

## Informe técnico

**Módulo:** Programación 3

**Carrera:** Taller de Desarrollo de VideoJuegos

**Fecha:** 26/6/2026

**Nombres:** Bastian Carracedo Jara y Ignacio leiva

**Sede:** CFT San Agustin



CFT SAN  
AGUSTÍN  
actitud profesional

## **Contextualización**

Este proyecto lleva el desarrollo de un sistema de RPG diseñado bajo los principios de la Programación Orientada a Objetos. El objetivo principal es la creación de un sistema escalable y desacoplado, donde la lógica de los personajes, el inventario y las interacciones del mundo operen como módulos independientes.

## Implementación Técnica

### Personajes

```
using UnityEngine;
namespace RPG.Characters
{
    3 referencias
    public interface IDamageable
    {
        4 referencias
        void RecibirDanio(float cantidad);
    }

    Script de Unity | 3 referencias
    public abstract class PlayerBase : MonoBehaviour, IDamageable
    {
        [Header("estadísticas")]
        public float maxVida = 100f;
        protected float vidaActual;

        public static event Action<float> OnVidaCambiada;

        Mensaje de Unity | 0 referencias
        protected virtual void Start()
        {
            vidaActual = maxVida;
        }

        Mensaje de Unity | 0 referencias
        protected virtual void Update()
        {
            if (Input.GetKeyDown(KeyCode.Q))
            {
                UsarHabilidadEspecial();
            }
        }

        4 referencias
        public abstract void UsarHabilidadEspecial();

        4 referencias
        public virtual void RecibirDanio(float cantidad)
        {
            vidaActual -= cantidad;
            Debug.Log(gameObject.name + "recibiste daño, vida:" + vidaActual);

            OnVidaCambiada?.Invoke(vidaActual);
        }
    }
}
```

*Código Principal que hace de padre dentro de la jerarquía*

## Interfaces

```
[Header("Configuración")]  
public float cantidadDanio = 20f;
```

📧 Mensaje de Unity | 0 referencias

```
private void OnTriggerEnter(Collider other)  
{
```

```
    IDamageable objetivo = other.GetComponent<IDamageable>();
```

```
using System.Collections;  
using System.Collections.Generic;  
using UnityEngine;  
using RPG.Inventory;
```

📧 Script de Unity (3 referencias de recurso) | 0 referencias

```
public class ItemRecoger : MonoBehaviour
```

```
{  
    public string nombreArma;  
    public int danoArma;  
    📧 Mensaje de Unity | 0 referencias  
    private void OnTriggerEnter(Collider other)  
    {
```

```
        if (other.CompareTag("Player"))  
        {
```

```
            InventarioArmas inventario = other.GetComponent<InventarioArmas>();
```

```
            if (inventario != null )  
            {
```

```
                inventario.AgregarArma(new Arma(nombreArma, danoArma));
```

```
                Debug.Log("Recogiste: " + nombreArma);
```

```
                Destroy(gameObject);  
            }  
        }  
    }
```

```
using System.Collections;  
using System.Collections.Generic;  
using UnityEngine;  
using RPG.Inventory;  
namespace RPG.UserInterface
```

```
public class UI : MonoBehaviour
```

```
{  
    void OnEnable()  
    {
```

```
        InventarioArmas.OnArmaEquipada += AIEquiparArma;
```

```
    }
```

```
    void OnDisable()  
    {
```

```
        InventarioArmas.OnArmaEquipada -= AIEquiparArma;
```

```
    }
```

```
    void AIEquiparArma(Arma arma)
```

```
    {
```

```
        Debug.Log("UI recibí el evento: se equipó " + arma.nombre);
```

```
    }
```

Estos interactúan con todos los que son colisiones y hacen accionar un trigger para estar de una forma aplicada en el momento que se le da su interacción

## inventario y eventos

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

namespace RPG.Characters
```

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

namespace RPG.Control
```

```
using System.Collections;
using System.Collections.Generic;
using System;
using UnityEngine;

namespace RPG.Characters
```

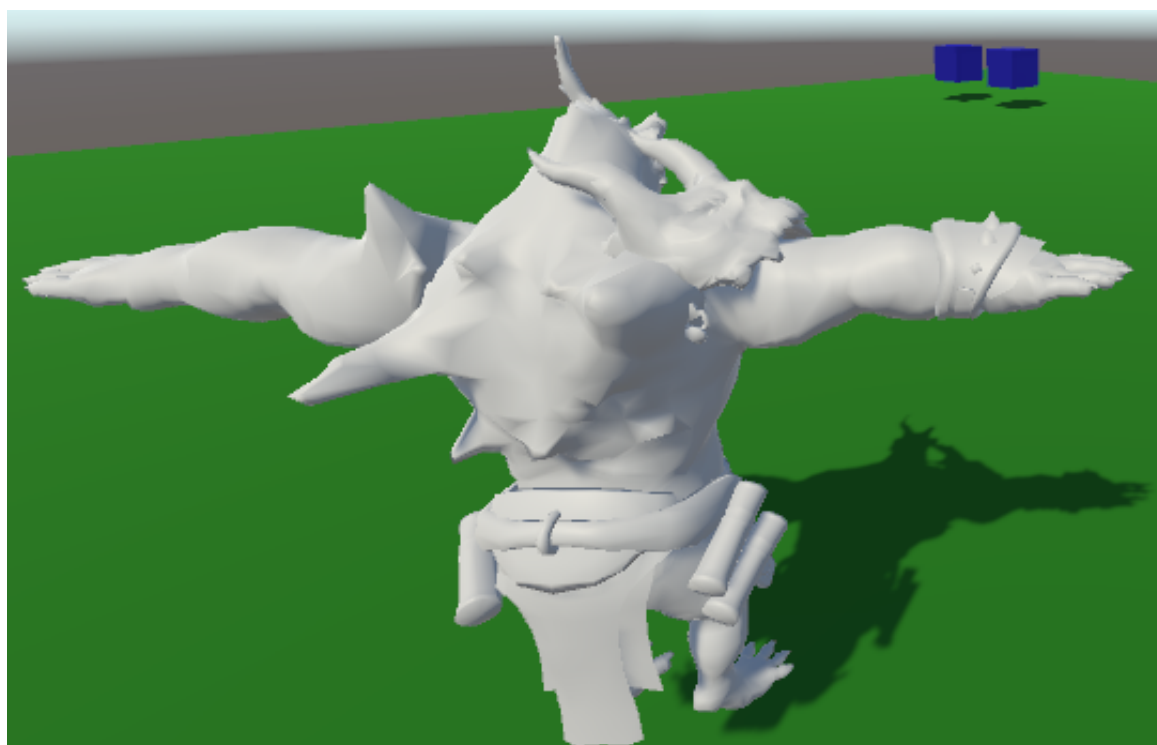
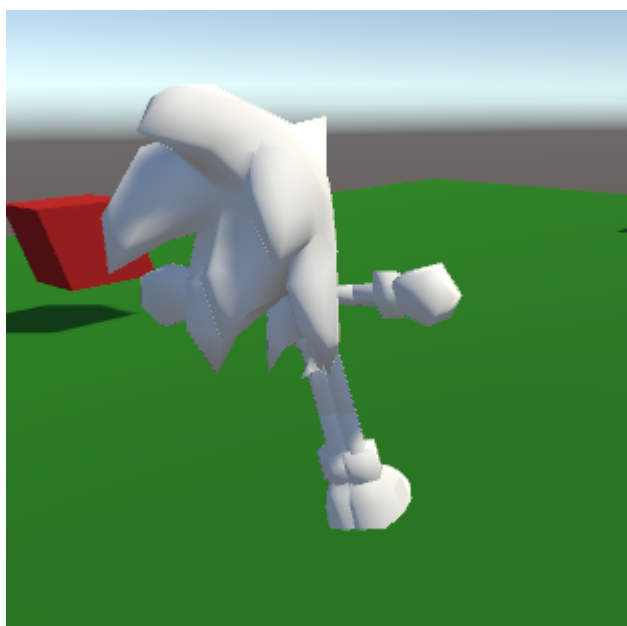
```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using RPG.Inventory;

namespace RPG.Inventory
```

## Diagnóstico de rendimiento (Unity Profiler):

| Fase      | FPS | CPU(ms) |
|-----------|-----|---------|
| Primitiva | 60  | 16ms    |
| Low Poly  | 250 | 4ms     |
| High Poly | 200 | 5ms     |

| Fase      | Nombre del Objeto  | Triángulos | Vértices |
|-----------|--------------------|------------|----------|
| Primitiva | Sonic              | ~80        | ~160     |
| Low Poly  | Shaun              | 742        | 644      |
| High Poly | Warrok W Kurniawan | 11.600     | 13.100   |



### **Análisis técnico de rendimiento**

La fase con mejor rendimiento fue Low Poly con 250 FPS y 4 ms de CPU, ya que el modelo Shaun cuenta con solo 742 triángulos, lo que genera una carga geométrica mínima para el hardware.

La fase con peor rendimiento fue Primitiva con 60 FPS y 16 ms de CPU. Esto se debe a que Unity limita los FPS a 60 por defecto con VSync activado, no por carga geométrica.

La fase High Poly con el modelo Warrok W Kurniawan de 11.600 triángulos obtuvo 200 FPS y 5ms de CPU, mostrando un leve aumento en la carga respecto al Low Poly pero manteniéndose eficiente.

En conclusión, a mayor cantidad de triángulos, mayor es el tiempo de procesamiento de la CPU, lo que se refleja directamente en los FPS obtenidos.